

Portfolio

유저의 니즈를 파악하고 소통에 중점을 두는 게임 클라이언트 개발자 **김종찬**입니다.

Unity 기반의 3D TPS 슈팅 게임 (PC, Mac 지원) 포트폴리오의 소개 내용을 담았습니다.

Chicken-Royale

치킨-로얄을 소개 합니다.

김종찬 KimJongChan

유니티 클라이언트 개발자

☎ +82 010-4114-4605

✉ github.com/Chan1605

📅 1996.05.16



Unity 기반 3D 게임 개발을 중심으로 프로젝트를 진행했습니다.

FSM 기반 AI 시스템, 인벤토리 시스템, PlayFab 서버 연동 등을 구현하며

게임 플레이 중심의 기능 개발을 경험했습니다.

문제를 구조적으로 분석하고 재사용 가능한 코드 설계를 중요하게 생각합니다.

Project 01

프로젝트 소개

Chicken Royale 기술 개발서

프로젝트명

Chicken Royale

주요목표

Unity 3D TPS Game 제작

개요

- ✓ 기간: 6개월
- ✓ 역할: 게임 시스템 및 클라이언트 개발
- ✓ 팀 구성: 1인 개발
- ✓ 목표: 개인 포트폴리오를 넘어선 스팀 출시

핵심파트

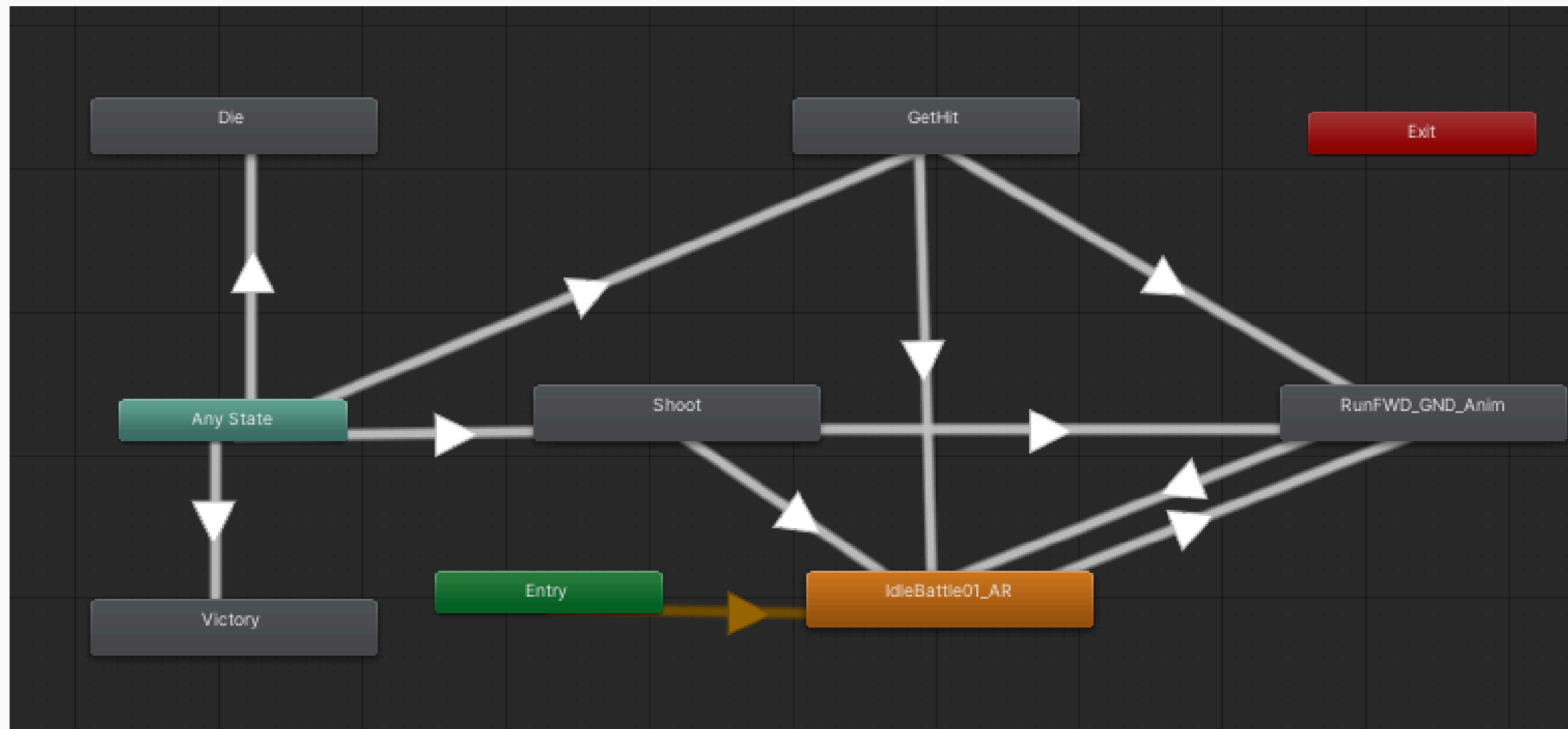
- ✓ Enemy AI (FSM)
- ✓ 인벤토리 시스템
- ✓ 랭킹 시스템 (PlayFab)

Enemy Animation (상태 다이어그램)

Project 02

적 상태 다이어그램 소개

- Any State 기반으로 상태머신을 설계 했습니다
- 유니티 애니메이터의 SetBool, SetTrigger 활용해서 적의 상태를 체크하여 전환합니다.



Enemy AI (FSM)

```
void MonStateUpdate()
{
    if (m_isDie)
        return;

    switch (m_MonSt)
    {
        case Monstate.idle:
            m_Anim.SetBool("isTrace", false);
            break;
        case Monstate.trace:
            {
                m_Anim.SetBool("isTrace", true);
                Vector3 m_MoveDir = Vector3.zero;
                m_MoveDir = m_target.transform.position - m_montr.transform.position;
                m_MoveDir.y = 0;

                Vector3 a_Monstep = (m_MoveDir.normalized *
                                     Time.deltaTime * m_monspeed);
                transform.Translate(a_Monstep, Space.World);

                if (0.1f < m_MoveDir.magnitude)
                {
                    Quaternion a_TargetRot;
                    float m_RotSpeed = 7.0f;
                    a_TargetRot = Quaternion.LookRotation(m_MoveDir);
                    transform.rotation =
                        Quaternion.Slerp(transform.rotation, a_TargetRot,
                                          Time.deltaTime * m_RotSpeed);
                }

                m_Anim.SetBool("isTrace", true);
            }
            break;
        case Monstate.attack:
            {
                m_curdelay += Time.deltaTime;
                if (m_curdelay < m_maxdelay)
                    return;
            }
    }
}
```

```
public void InitMonster()
{
    m_CurHp = m_MaxHp;          // 체력 초기화
    m_isDie = false;
    m_MonSt = Monstate.idle;    // 기본 상태로

    HpbarCtrl hpbarCtrl = FindObjectOfType<HpbarCtrl>();
    hpbarCtrl.Register(this.transform, this);
    hpBar.GetComponent<Image>().fillAmount = 1.0f;

    switch (m_DeathCount)
    {
        case >= 2:
            m_EnemyBulletIndex = 2;
            m_maxdelay = 0.1f;
            m_Eattackdamage = 20;
            m_tracedist = 100f;
            m_attackdist = 20f;
            m_monspeed = 10f;
            transform.localScale = Vector3.one * 2.5f;
            ChangeToDeadMaterials();
            break;
        case >= 1:
            m_EnemyBulletIndex = 1;
            m_maxdelay = 0.2f;
            m_Eattackdamage = 10;
            m_tracedist = 50f;
            m_monspeed = 8f;
            m_attackdist = 17f;
            //transform.localScale = Vector3.one * 2.5f;
            ChangeTintColor(Color.red);
            break;
        default:
            m_EnemyBulletIndex = 0;
            m_maxdelay = 0.3f;
            m_Eattackdamage = 1;
            break;
    }
}
```

Enemy AI 코드 소개

02

FSM을 기반으로 설계 하였습니다. (좌측)

- State
- Idle (평소)
- Trace (추적)
- Attack (플레이어 발견 후 공격)

Update 함수에서 상태를 체크하고
조건에 따라 상태 전환이 이루어지도록 설계했습니다.

initMonster 함수 설계 내용입니다. (우측)

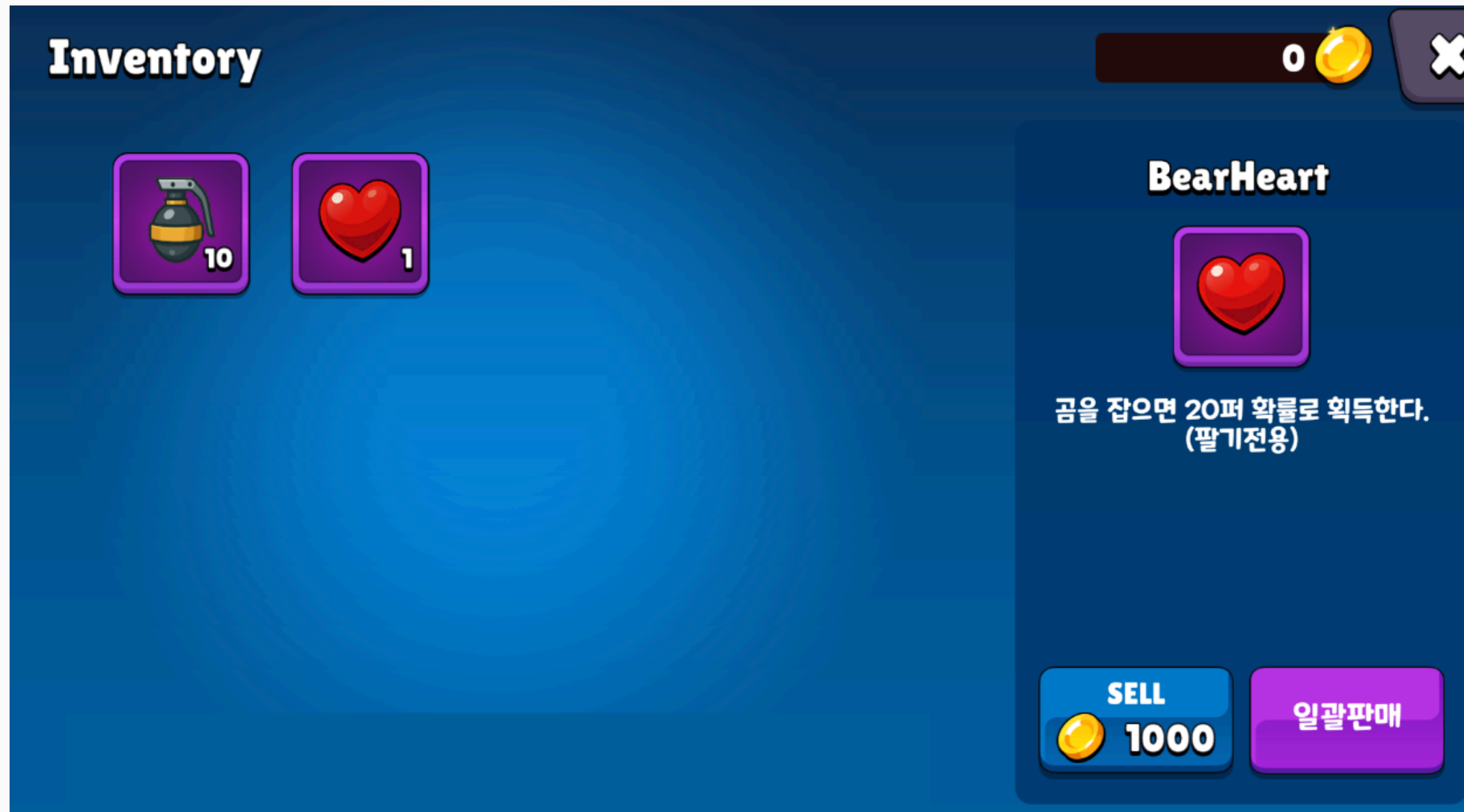
- 적이 사망할 때 마다 능력치가 강화 되도록 설계 하였습니다.
- 적이 사망할 때 마다 플레이어가 강해진 적을 인식 할 수 있도록 마테리얼 색상을 변경하였습니다.
- 마지막으로 강해진 적은 크기가 변경 되도록 설계 했습니다.

Project 03

Inventory

적 사망 시 재화 획득

인벤토리 기능 소개



주요 목적

적을 잡으면 아이템이 획득 되고
저장 되도록 설계

구현 기능

아이템 관리 및 사용 기능을 포함한
인벤토리 시스템을 구현했습니다.

- 아이템 획득
- 아이템 사용
- 아이템 판매
- UI 연동

Inventory

```
0 references
public enum ItemType
{
    0 references
    Yellow,
    1 reference
    Green,
    1 reference
    Purple,
    0 references
    Blue,
    1 reference
    Brown
}

5 references
public enum ItemBehaviorType
{
    1 reference
    Use,
    1 reference
    Sell,
    2 references
    None
}

[CreateAssetMenu(fileName = "NewItem", menuName = "Inventory/Item")]
19 references
public class InventoryItemData : ScriptableObject
{
    10 references
    public string itemId;
    1 reference
    public string itemName;
    4 references
    public Sprite icon;
    12 references
    public int quantity;
    3 references
    public ItemType itemType;
    1 reference
    public string description;
}
```

```
public class InventoryManager : MonoBehaviour
{
    17 references
    public static InventoryManager Inst;
    15 references
    private readonly Dictionary<string, InventoryItemData> inventory = new();
    6 references
    private readonly Dictionary<string, InventorySlotUI> slotUIs = new();
    [Header("----- UI -----")]
    3 references
    [SerializeField] private Transform slotParent;
    2 references
    [SerializeField] private GameObject slotPrefab;
    1 reference
    public TextMeshProUGUI[] CurGold;

    0 references
    void Awake()
    {
        if (Inst == null)
            Inst = this;
        else
            Destroy(gameObject); // 중복 방지
    }

    1 reference
    public void AddGold(int amount)
    {
        GameMgr.inst.AddGold(amount);
        UpdateGoldUI();
    }

    3 references
    public void UpdateGoldUI()
    {
        foreach (var text in CurGold)
        {
            if (text != null)
                text.text = GameMgr.inst.Gold.ToString("N0");
        }
    }

    5 references
    public void AddItem(InventoryItemData newItem, int amount = 1)
    {
    }
}
```

Project 03

Inventory 코드 소개

Inventory 구조 다이어그램

└ ItemData
└ ItemSlot
└ UI

item Data (좌측)

- 유니티 에디터에서 직접 아이템을 등록한 후 등급별로 획득 할 수 있도록 설계 했습니다.

Inventory Manager (우측)

- 전체 인벤토리를 담당하고 유저가 아이템을 사용하면 획득 및 차감되는 UI/UX를 설계 했습니다.

Project 04

PlayFab

로그인 서버 연동

로그인 기능 소개

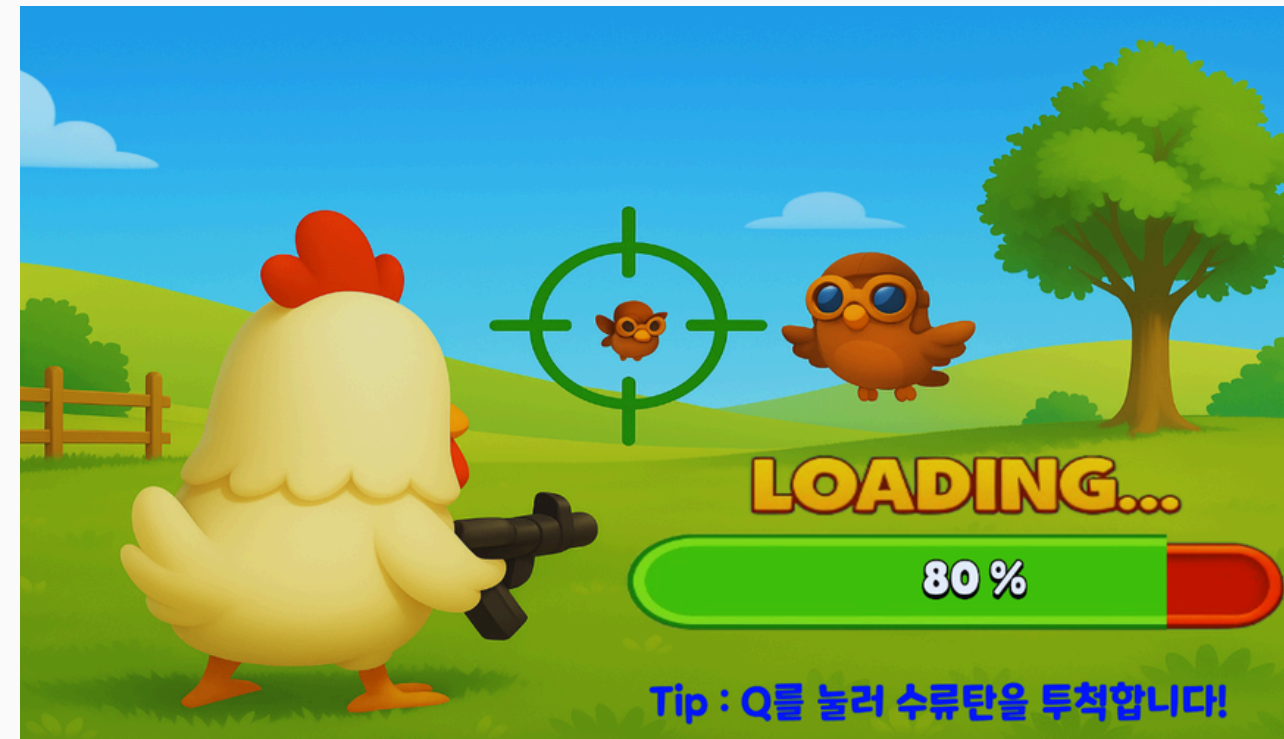
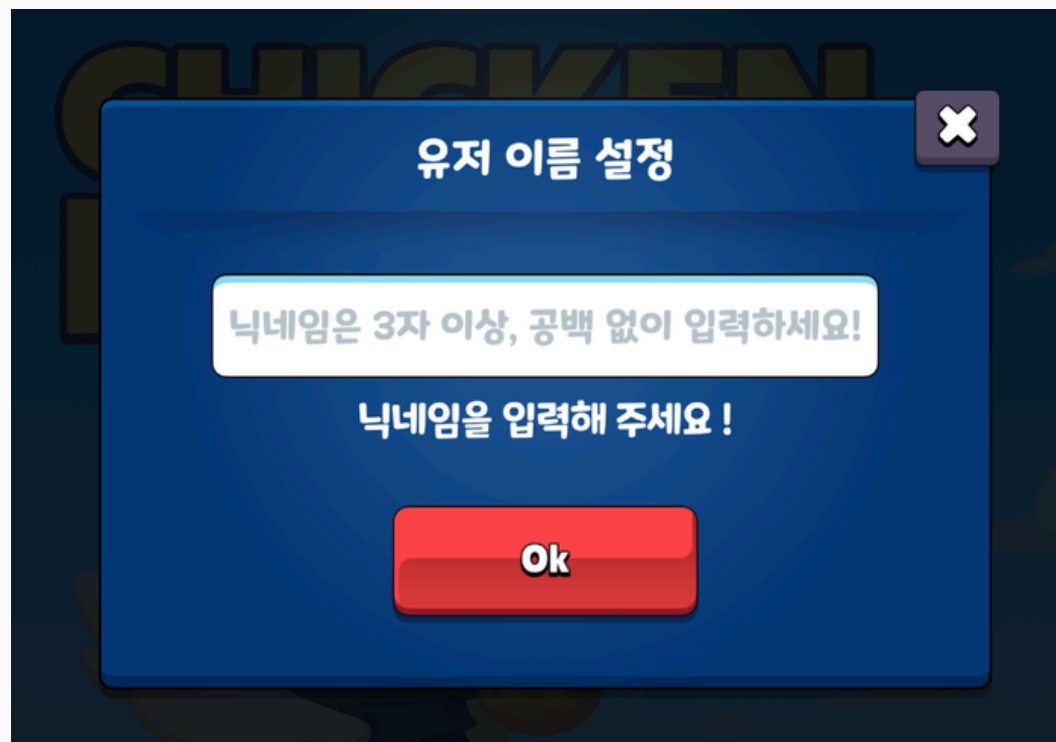
주요 목적

유저가 원하는 닉네임 설정 할 수 있도록 설계

구현 기능

PlayFab을 이용하여 유저 닉네임을 서버에 저장했습니다.

- 중복 닉네임 체크
- 최소,최대 닉네임 체크
- 공백(스페이스 바) 체크
- 로그인 유효성 체크 후 로딩씬으로 진입



Project 04

PlayFab

랭킹 서버 연동

랭킹 기능 소개



Type	Name	Last login ↓
Player	AB44D5CCA8ED7DFE 김종찬	5:28 AM
Player	50EAA23EEA887976 히히히	Mar 5, 2026 3:43 AM
Player	B90E460167D9E20F Dfa	Aug 13, 2025 10:20 AM
Player	375ABB330DB925CE Wwww	Jul 19, 2025 1:24 PM
Player	E9D9B3D9A355A177 선정이다	Jul 17, 2025 4:36 AM
Player	3989742C855ADA1C 김김김김김	Jul 7, 2025 11:55 AM
Player	61851E5804CF026F	Jul 7, 2025 11:55 AM

주요 목적

플레이어 점수를 서버에 저장하여
유저 간 순위 비교가 가능한 랭킹 시스템을 설계

구현 기능

PlayFab을 이용하여 최고점수,
킬 카운트를 서버에 저장했습니다.

- 최대 10등까지만 랭킹 패널(UI)에 표시
- Top3까지 메달 표시
- 플레이어가 사망 시 최고 점수 업데이트

Name	Value
HighKill	49
HighScore	11100
+ Add	

PlayFab 서버 연동

```
2 references
public void LoginAndSetDisplayName(string displayName, System.Action onComplete = null)
{
    PlayFabSettings.TitleId = "1EF77A";

    string customId;
    if (PlayerPrefs.HasKey("MyCustomId"))
    {
        customId = PlayerPrefs.GetString("MyCustomId");
    }
    else
    {
        customId = System.Guid.NewGuid().ToString();
        PlayerPrefs.SetString("MyCustomId", customId);
    }

    PlayFabClientAPI.LoginWithCustomID(new LoginWithCustomIDRequest()
    {
        CreateAccount = true,
        CustomId = customId
    },
    success =>
    {
        Debug.Log("PlayFab 로그인 성공");

        IsLoggedIn = true;
        CheckAccountValidity();

        if (success.NewlyCreated)
        {
            Debug.Log("신규 계정이 생성되었습니다.");
        }

        SetDisplayName(displayName, onComplete);
    },
    failure =>
    {
        Debug.LogError("PlayFab 로그인 실패: " + failure.GenerateErrorReport());
    }
}
```

```
void UpdateRankPanelInfo()
{
    if (PlayFabLogin.IsLoggedIn)
    {
        Debug.Log($"로그인 상태: {PlayFabLogin.IsLoggedIn}");
        // 닉네임 불러오기
        PlayFabClientAPI.GetAccountInfo(new GetAccountInfoRequest(),
        result =>
        {
            string nickname = result.AccountInfo.TitleInfo.DisplayName;
            m_Rankname.text = $"{nickname}";
        },
        error =>
        {
            Debug.LogError("닉네임 불러오기 실패: " + error.GenerateErrorReport());
            m_Rankname.text = $"Unknown";
        });

        // 점수 불러오기
        PlayFabClientAPI.GetPlayerStatistics(new GetPlayerStatisticsRequest(),
        result =>
        {
            var stat = result.Statistics.Find(s => s.StatisticName == "HighScore");
            int highScore = stat != null ? stat.Value : 0;

            var highKillStat = result.Statistics.Find(s => s.StatisticName == "HighKill");
            int highKill = highKillStat != null ? highKillStat.Value : 0;

            m_RankScore.text = $"Kill Count : {highKill}\n {highScore}";
        },
        error =>
        {
            Debug.LogError("점수 불러오기 실패: " + error.GenerateErrorReport());
            m_RankScore.text = "0";
        });

        PlayFabClientAPI.GetLeaderboardAroundPlayer(new GetLeaderboardAroundPlayerRequest
        {
            StatisticName = "HighScore",
            MaxResultsCount = 1
        },
        result =>
        {
            m_RankLeaderboard.Clear();
            foreach (var player in result.Players)
            {
                m_RankLeaderboard.Add(player);
            }
        },
        error =>
        {
            Debug.LogError("리더보드 불러오기 실패: " + error.GenerateErrorReport());
        });
    }
}
```

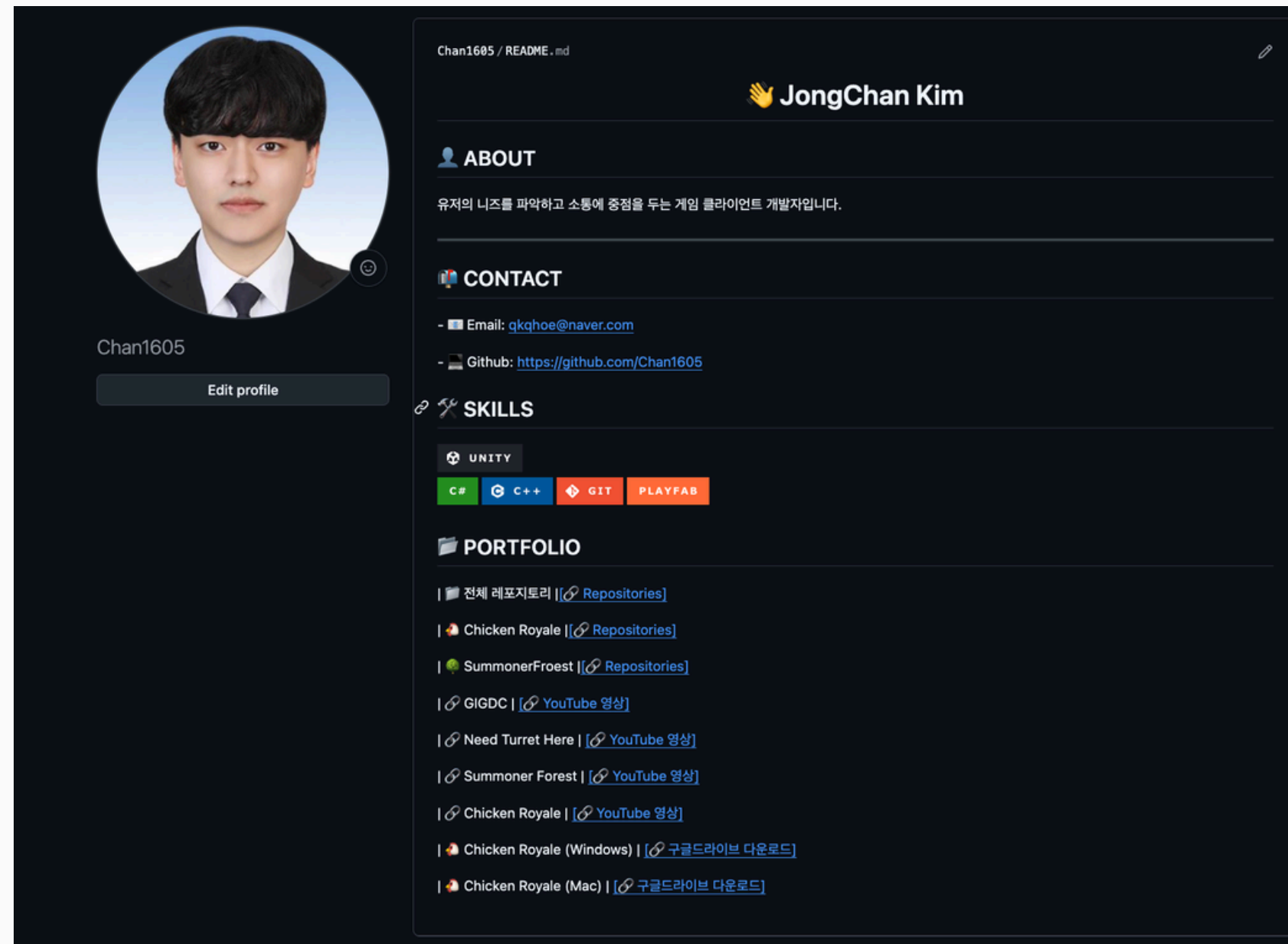
Project 04

PlayFab 코드 소개

로그인 및 랭킹 시스템

- 주요 기능
 - 로그인 (좌측)
 - 최고 점수 저장 및 랭킹 등록 (우측)

GitHub 링크



<https://github.com/Chan1605/Chicken-Royale>
프로젝트 코드 및 README에서
시스템 구조를 자세히 확인할 수 있습니다.



Thank you

함께 더 나은 게임 플레이를 위해.

제 포트폴리오를 봐주셔서 감사합니다.

지속적으로 시스템 설계 능력을 향상시키며 게임 플레이 경험을 개선하는 개발자가 되겠습니다.

